

INTRO TO SCIENTIFIC COMPUTING IN PYTHON

LECTURE 5

Zan Ahmad

Department of Applied Mathematics and Statistics,
Johns Hopkins University

January 16, 2026

Part I

INTRODUCTION TO MACHINE LEARNING

WHY MACHINE LEARNING APPEARS IN SCIENTIFIC COMPUTING

In scientific computing, we traditionally:

- ▶ write models from physical laws,
- ▶ simulate systems forward,
- ▶ analyze outputs.

However, many modern problems involve:

- ▶ large datasets,
- ▶ unknown or partially known models,
- ▶ noisy measurements.

Machine learning addresses this gap by learning models directly from data.

WHAT IS MACHINE LEARNING?

At its core, machine learning asks:

Can we infer a function from data?

Mathematically:

Given data $\{(x_i, y_i)\}$, find a function f such that $f(x_i) \approx y_i$

This is not fundamentally different from numerical approximation — it is a generalization of it.

TWO MAJOR LEARNING PARADIGMS

Machine learning problems fall into two broad categories:

Supervised learning

- ▶ data includes input-output pairs (x, y) ,
- ▶ goal: predict y from x .

Unsupervised learning

- ▶ data includes inputs x only,
- ▶ goal: discover structure in the data.

EXAMPLES

Supervised learning examples

- ▶ regression (predicting a continuous value),
- ▶ classification (predicting categories),
- ▶ system identification.

Unsupervised learning examples

- ▶ dimensionality reduction,
- ▶ clustering,
- ▶ feature discovery.

We will study:

- ▶ linear regression (supervised),
- ▶ principal component analysis (unsupervised).

LINEAR REGRESSION: PROBLEM SETUP

Suppose we observe:

$$(x_i, y_i), \quad i = 1, \dots, n$$

We assume a linear model:

$$y \approx w^T x + b$$

Goal:

- find parameters w and b that best fit the data.

MATRIX FORMULATION

Stack inputs into a matrix:

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

The model becomes:

$$y \approx Xw$$

This is an overdetermined linear system.

LEAST SQUARES OBJECTIVE

We choose w to minimize squared error:

$$J(w) = \|Xw - y\|_2^2$$

This is a convex optimization problem with a unique solution (if $X^T X$ is invertible).

DERIVATION VIA CALCULUS

Compute the gradient:

$$\nabla_w J(w) = 2X^T(Xw - y)$$

Set gradient to zero:

$$X^T Xw = X^T y$$

These are the **normal equations**.

CLOSED-FORM SOLUTION

If $X^T X$ is invertible:

$$w = (X^T X)^{-1} X^T y$$

Key insight:

Linear regression is linear algebra + calculus.

LINEAR REGRESSION IN PYTHON (NUMPY)

```
import numpy as np

X = np.column_stack((np.ones(n), x))
w = np.linalg.solve(X.T @ X, X.T @ y)
```

This uses:

- ▶ matrix multiplication,
- ▶ linear system solving,
- ▶ numerical stability.

WHY DIMENSIONALITY REDUCTION?

High-dimensional data:

- ▶ is expensive to store,
- ▶ is hard to visualize,
- ▶ often contains redundancy.

Principal Component Analysis (PCA) finds:

The directions of maximum variance in the data.

GEOMETRIC VIEW OF PCA

Given centered data:

$$X \in \mathbb{R}^{n \times d}$$

PCA finds orthogonal directions:

$$v_1, v_2, \dots$$

such that:

- ▶ v_1 captures the most variance,
- ▶ v_2 captures the next most variance,
- ▶ and so on.

COVARIANCE MATRIX

Compute covariance:

$$C = \frac{1}{n} X^T X$$

This matrix measures:

- ▶ variance along coordinates,
- ▶ correlation between coordinates.

EIGENVALUE PROBLEM

PCA solves:

$$Cv = \lambda v$$

- ▶ eigenvectors give principal directions,
- ▶ eigenvalues give variance explained.

PCA IN PYTHON (FROM SCRATCH)

```
Xc = X - X.mean(axis=0)  
C = Xc.T @ Xc / Xc.shape[0]
```

```
vals, vecs = np.linalg.eigh(C)
```

Sort eigenvectors by descending eigenvalue to get principal components.

PROJECTING ONTO PRINCIPAL COMPONENTS

Reduce dimension:

$$Z = Xv_k$$

This produces:

- ▶ lower-dimensional representation,
- ▶ maximal variance retention,
- ▶ orthogonal features.

WHAT YOU SHOULD TAKE AWAY

Machine learning is not magic.

It is:

- ▶ linear algebra,
- ▶ calculus,
- ▶ optimization,
- ▶ numerical algorithms.

Key message:

Machine learning is scientific computing applied to data.

WHERE TO GO NEXT

You now have the tools to:

- ▶ understand ML algorithms mathematically,
- ▶ implement them without black boxes,
- ▶ reason about numerical stability and scaling.

Everything beyond this course builds on these foundations.